

Project Incrementation Report — Week 2

What changed

- Connected Express to PostgreSQL and built the read endpoints. GET /api/players, GET /api/players/:id, GET /api/seasons, GET /api/matches (with an optional ?seasonId= filter), and GET /api/matches/:id all query the real Supabase Postgres database through pg now, instead of the app reading from hardcoded mock data.
- Built the match CRUD API. POST /api/matches, PATCH /api/matches/:id, and DELETE /api/matches/:id were added, with validation on the server (scores can't be negative or tied, the two players have to be different and actually exist, dates have to be real calendar dates). The server also refuses to accept a winnerId sent from the client — it always calculates the winner itself from the scores.
- Connected the React forms to the real API. Record Match and Edit Match now send their form data to the Express endpoints above instead of updating an in-memory mock array. Match History's delete button now calls the real DELETE endpoint.
- Made the leaderboard and player stats calculated, not stored. GET /api/leaderboard?seasonId= and GET /api/players/:id/stats calculate matches played, wins, losses, win percentage, rank, and current streak directly from the rows in the matches table. Both endpoints share the same calculation code (server/src/utils/leagueStats.js) so they can't disagree with each other.
- Swapped the frontend's data layer. src/services/mockApi.js and src/data/mockData.js were deleted and replaced with src/services/api.js, which makes real fetch() calls. All five pages (Leaderboard, Match History, Record Match, Edit Match, Player Profile) were switched over to use it.
- Fixed error messages so they're actually useful. Record Match and Edit Match used to always show a generic "could not save" message no matter what went wrong. That was changed so the real error message from the backend (like "Scores cannot be tied") shows up instead.

Responsive layout was not part of this week's work — that was already built earlier (Phase 2.5) and just kept working through this integration without needing changes.

Why

Up to this point the app was a frontend that only looked complete — it was working off fake, in-memory data, so nothing actually persisted and the "leaderboard" was really just a JavaScript function running on whatever fake matches existed in memory. The whole point of Barkada League is that the leaderboard reflects real recorded matches

automatically, so that only works if there's a real database behind it and a real API in between that the frontend can't just skip past.

The other reason the API needed to own the winner calculation is that if the frontend could send `winnerId` directly, the "leaderboard is derived from real match data" idea falls apart — anyone could just tell the server whatever winner they wanted. Making the server calculate it from the scores every time, on create and on edit, keeps that meaningful.

What broke or what I got stuck on

- Env file mix-up. I have two `.env` files (one for the frontend, one for the backend) and at one point pasted the real Supabase connection string into the wrong one — the tracked `.env.example` instead of the gitignored `.env`. Caught it before committing either time and fixed it by moving the real value to the correct file.
- Postgres date/timezone issue. Dates coming back from the database were shifting by a day depending on the server's local timezone. This happened because the pg driver was converting a plain DATE column into a JavaScript Date object at local midnight instead of keeping it as a date string. Fixed by casting date columns to `::text` in the SQL queries so pg just returns the string.
- A crash that only showed up with real data. Once real API data was flowing in, the leaderboard page crashed for any player with zero matches, because a line of code read `currentStreak.type` without checking whether `currentStreak` could be null first. The mock data I'd been testing against always had every player with at least one match, so this bug existed the whole time without me ever seeing it. One-line fix using optional chaining (`?.`) once I found it.
- Git push failing. Separately from the app itself, pushing to GitHub started failing with an HTTP 400 error. This turned out to be a known git/HTTP2 issue with the local git config, not a problem with the repo or the code — fixed by changing `http.version` and `http.postBuffer` in git config. Mentioning it here since it blocked me for a bit, but it's a tooling issue, not an application bug.

What is left

The application functionality itself is done. What's left is finalization/submission work:

- Final review of the documentation (README, this report, the security checklist, AI usage) to make sure it all matches the actual repo.
- Double-checking the security checklist answers are still accurate.
- Confirming the live database is still exactly 6 players, 1 season, and 15 matches, with no leftover test records from development.
- Final pass over the screenshots to make sure they still match the current app.
- Preparing for the final presentation.